Prototype

Season: 1 · Creational · Category: GoF · Level: Beginner

Intent

- Clone a preconfigured object instead of re-building it.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- Photocopying a filled form and changing only the name.

When to Use

- Costly setup; many similar instances.
- The design force named in Intent shows up more than once.

When Not to Use

- Cheap constructors; prefer factories.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.gof.creational.prototype;

import java.util.ArrayList;
import java.util.List;

/** Prototype: clone a preconfigured object instead of rebuilding from scratch. */
public final class DocumentTemplate implements Cloneable {
    private String title;
    private final List<String> sections;

    public DocumentTemplate(String title, List<String> sections) {
        this.title = title;
        this.sections = new ArrayList<>(sections);
    }

    public String title() { return title; }
    public void setTitle(String title) { this.title = title; }
    public List<String> sections() { return sections; }

    /** Deep copy so mutating the clone does not affect the prototype. */
    @Override
    public DocumentTemplate clone() {
        return new DocumentTemplate(title, sections);
    }
}

/** Modern: records are naturally copyable via canonical constructor / with-style helpers. */
record SlideDeck(String title, List<String> slides) {
    SlideDeck {
        slides = List.copyOf(slides);
    }
```

Classic Implementation (2)

```
    SlideDeck customized(String newTitle) {
        return new SlideDeck(newTitle, slides);
    }
}
```

Modern Java 25 / Functional

- SlideDeck record with customized() copy.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: gof/creational/prototype/
```

Pitfalls

- Deep vs shallow copy.
- Records + List.copyOf simplify immutability.

Interview Q&A

- Q: Cloneable pitfalls? A: Cloneable is broken-by-design; prefer copy constructors or records.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Clone a preconfigured object instead of re-building it.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: Photocopying a filled form and changing only the name. [0:40] Intent — State the design force in one...